

Mulberry for macOS v1 Product Requirements

Native Desktop Client, Passive Presence Overlay, and Quick Draw

Mulberry Product and Engineering

May 2026

Abstract

Mulberry for macOS v1 extends Mulberry from an Android-only paired canvas into a first-class cross-platform, multi-device product. The macOS client must let an existing paired user sign in with Google, see the same private pair canvas, draw the same canvas operations, send the same lightweight reactions, and participate equally with all of the user's Android devices and the partner's devices. The defining macOS addition is a desktop presence layer: a passive, click-through, borderless overlay that sits visually on the desktop below normal windows, plus a temporary Quick Draw mode raised by a global shortcut for fast drawing without opening the full app. This document fixes the v1 scope, user experience, acceptance criteria, non-goals, and a modular implementation plan.

1 Context and Product Thesis

Mulberry's current product value is ambient one-to-one connection rather than collaboration. Two people share a private canvas; small drawings, text, stickers, and reactions appear quietly in the other person's environment. Android expresses this primarily through a live wallpaper and a foreground drawing app. macOS should preserve the same emotional contract while using platform-native surfaces: menu bar presence, a transparent desktop overlay, a full SwiftUI app window, and a temporary Quick Draw mode.

The v1 macOS app is not a companion viewer. It is a real Mulberry client. A paired user who signs in on Mac participates in the same pair session as their Android phone. If the user draws on the Mac, the drawing appears on their phone, on their partner's phone, and on any other live clients. If the partner draws on Android, the Mac updates through the same server-assigned revision stream.

1.1 Product Principles

1. **One pair, many devices.** Pairing remains between two users, not two physical devices. Any authenticated device owned by either paired user can observe and mutate the pair canvas.
2. **Ambient by default.** The passive overlay is quiet, click-through, and below normal windows. It should not become desktop clutter or block work.
3. **Fast expression when invoked.** Quick Draw must be available through Command-Control-M and must become interactive immediately.
4. **Parity over reinvention.** The first macOS release supports the Android canvas operation set: strokes, text elements, stickers, clear, undo/redo semantics where available, reactions, profile/pair status, and streak display.
5. **Public macOS APIs only.** The product must not rely on private Finder layering, wallpaper injection, or undocumented fullscreen behavior.

6. **Architecture before breadth.** v1 should land as a maintainable native codebase with clean sync, persistence, rendering, and windowing seams, even if some catalog/admin polish waits.

2 Users and Use Cases

2.1 Primary User

The primary user is an existing Mulberry user who is already paired through Android and wants their Mac to become another device in the same relationship. The user expects the Mac app to show the same canvas, use the same account, respect the same pair relationship, and never require a new pair flow when the account is already paired.

2.2 Primary Jobs

Job	Required v1 behavior	Surface
See latest canvas	The passive overlay shows the latest locally rendered canvas content and updates when sync receives new operations.	Desktop overlay
Draw quickly	Command-Control-M enters Quick Draw, raises the overlay, shows tools, accepts drawing input, syncs operations live, and exits cleanly.	Overlay + tool palette
Use full app	The user can open a full app window for drawing, text, stickers, settings, account, pairing status, and troubleshooting.	Main app window
Send affection	The user can send a heart/reaction from the menu bar and full app.	Menu bar, app
Understand status	The user can see connection state, last sync, pair status, stale state, and streak.	Menu bar, overlay, app
Recover after restart	When launched at login or manually re-opened, the Mac catches up from the last applied server revision.	Background client

3 V1 Scope

3.1 Included

- Native macOS app written in Swift and SwiftUI, with AppKit where required for windowing and hotkeys.
- Direct DMG distribution for v1. The implementation should avoid choices that make later App Store packaging impossible.
- Google Sign-In only. Sign in with Apple is out of scope.
- Status bar app with menu items for Open Mulberry, Quick Draw, Send Heart, Streak, Connection Status, Settings, and Quit.
- Login item support using the public Service Management API.

- Passive overlay on one selected display, across all Spaces, below normal windows, click-through, borderless, transparent, and shadowless.
- Quick Draw mode raised by Command-Control-M through a registered global hotkey.
- Quick Draw temporarily raises the overlay and tool palette above normal app windows, accepts input, and exits on Done, Escape, app deactivation, or explicit menu action.
- Full app drawing surface with feature parity for Android canvas operations.
- Local persistence for account state, pair state, canvas snapshot, operation ledger, pending outbound operations, sticker asset metadata, and user settings.
- WebSocket sync using the existing canvas protocol where possible, plus REST recovery.
- Visible notifications aligned with Android notification categories where applicable.

3.2 Explicit Non-Goals

- No private wallpaper injection or direct drawing into the macOS wallpaper layer.
- No guarantee that the passive overlay appears above desktop icons or macOS desktop widgets.
- No Quick Draw over another app’s fullscreen Space in v1. If the user invokes Quick Draw from a fullscreen Space, the app opens a focused prompt instructing them to leave fullscreen or open the full app.
- No iOS/iPadOS client in this project phase.
- No Sign in with Apple in v1.
- No App Store submission in v1, though architecture must remain compatible.
- No public multi-user collaboration, groups, feeds, comments, or sharing.
- No end-to-end encryption migration in v1.

4 Experience Requirements

4.1 Application Shell

The macOS app launches as an accessory-style app with a status bar extra and no required Dock icon in normal operation. On first launch it may show a full app window for sign-in and onboarding. Once authenticated, the status bar item remains the always-available control point. The user can open the full app window at any time.

The status menu must include, in order:

1. Pair status line: partner name or “Not paired”.
2. Sync status line: Connected, Connecting, Offline, Recovering, or Error.
3. Open Mulberry.
4. Quick Draw, with Command-Control-M shown.
5. Send Heart.
6. Streak summary.
7. Overlay submenu: Show/Hide Overlay, Choose Display, Reset Position.
8. Settings.

9. Quit Mulberry.

4.2 Passive Overlay

The passive overlay is a read-only consumption surface. It is not a normal editing surface and must not capture mouse or keyboard input. It renders only the current canvas content and minimal status affordances. It must not render a permanent opaque card background behind the canvas content. The window background is transparent; canvas content is full opacity.

The overlay must:

- use a borderless AppKit panel/window;
- use clear background and non-opaque rendering;
- set its window level below normal app windows;
- ignore mouse events in passive mode;
- join all Spaces;
- stay on exactly one selected display;
- persist frame and display selection;
- avoid covering desktop interaction.

4.3 Quick Draw

Quick Draw is a temporary edit mode for the overlay, not an always-floating preference. It is entered by Command-Control-M, the status menu, or an optional full app button. It exits through Done, Escape, app deactivation, and explicit status menu action.

On entry:

1. If the current Space belongs to another app's fullscreen window, show a prompt instead of attempting to float over it.
2. Activate Mulberry.
3. Raise the overlay to a floating level.
4. Enable mouse and keyboard input for the overlay.
5. Show a compact tool palette adjacent to the overlay.
6. Focus the overlay so Escape and keyboard commands work.

On exit:

1. Commit any active stroke.
2. Hide tool palette.
3. Return overlay to passive low level.
4. Set the overlay to ignore mouse events.
5. Restore non-intrusive accessory behavior.

4.4 Full App

The full app is the canonical place for rich editing, account flows, sticker browsing, settings, and troubleshooting. It must use native SwiftUI with AppKit bridges only where needed. It should

visually align with Mulberry’s warm, playful canvas identity without becoming a marketing surface. The first screen after sign-in is the live canvas, not a landing page.

The final visual design will be provided later. Until then, engineering must build a barebone, functional UI that achieves feature completeness without inventing a divergent desktop product language. The default visual and interaction reference is the existing mobile app: tool grouping, canvas-first hierarchy, pairing/profile surfaces, sticker behavior, and notification/settings semantics should aim for parity unless a macOS platform convention clearly requires adaptation.

4.5 Canvas Tools

V1 must support:

- freehand drawing with color and brush width;
- eraser behavior consistent with Android semantics;
- clear canvas with confirmation in full app and a guarded command in Quick Draw;
- undo/redo for local user actions using operation-aware semantics;
- text elements with content, color, background pill, font, alignment, center, rotation, scale, and box width;
- sticker elements with pack key, version, sticker ID, center, rotation, and scale;
- passive rendering of partner operations;
- visible sync state for pending, accepted, recovered, and failed operations.

5 Backend and Cross-Platform Requirements

Pairing remains user-to-user. Device registration must become platform-aware. The backend currently models Android push tokens and canvas operations; macOS v1 requires device records that can represent macOS, APNs token readiness later, app version, app environment, and device instance identity.

5.1 Required Server Behavior

- Existing Google auth remains the authentication mechanism.
- Existing access and refresh token model may be reused by macOS.
- The bootstrap response must be sufficient for macOS to know auth state, pair state, partner profile, streak, overlay eligibility, and latest canvas revision.
- The canvas WebSocket must allow multiple simultaneous connections for the same user within the same pair session.
- Operation acceptance must remain idempotent by client operation ID.
- Recovery endpoints must return all operations after a revision and a snapshot fallback.
- Device token registration should evolve from Android-only to platform-aware, but APNs silent push is not required for v1 launch if the login item is running.

6 Functional Acceptance Criteria

Area	Acceptance criteria
Sign-in	A new Mac install can authenticate with Google, store tokens securely, refresh tokens, sign out, and recover from expired access tokens.
Bootstrap	A paired user lands directly in the live canvas state; an unpaired user sees the same account/pairing state semantics as Android.
Passive overlay	The overlay is visible on the selected desktop when uncovered, below normal windows, click-through, transparent around content, and present across Spaces on the selected display.
Quick Draw	Command-Control-M opens Quick Draw from another normal app; overlay raises, accepts drawing, shows tools, and returns passive on Escape or Done.
Sync	Drawing on Mac appears on Android and partner devices; drawing on Android appears on Mac without requiring manual refresh.
Recovery	Killing and reopening the app recovers missed operations from last applied revision.
Persistence	Restart preserves user session, pair state, latest canvas, pending outbound queue, overlay position, selected display, and preferences.
Notifications	User-visible notifications match Android intent for reactions, pair changes, and meaningful canvas nudges where supported.
Failure handling	Offline state is visible, local edits queue safely, and recovery resumes without duplicate operations.

7 Implementation Plan

The implementation is divided into twelve phases. Each phase must be shippable behind a development flag or independently testable. Do not merge phases that cross storage, sync, rendering, and platform behavior unless the tests for both are already in place.

7.1 Phase 1: macOS Workspace Foundation

Create the permanent macOS app target, package layout, code signing placeholders, build scripts, and CI smoke target. Replace the POC executable with a real app bundle target. Establish module boundaries for AppShell, Auth, Networking, Sync, CanvasCore, CanvasRendering, Persistence, Overlay, QuickDraw, Notifications, and Settings.

Acceptance: the app builds locally, launches as an accessory app, shows a status bar item, opens a blank full app window, and has a documented development run command.

7.2 Phase 2: App Shell, Menu Bar, and Window Routing

Implement status bar menu, full app window coordinator, app activation policy, single-instance behavior, and basic settings window. The status menu should show hardcoded status until real services land.

Acceptance: menu bar commands open the app window, show/hide overlay, quit cleanly, and never create duplicate primary windows.

7.3 Phase 3: Overlay and Quick Draw Platform Layer

Productionize the POC windowing model. Implement passive low-level overlay, display selection, frame persistence, all-Spaces behavior, click-through mode, Quick Draw raising, tool palette, Escape handling, and Command-Control-M hotkey registration with failure detection.

Acceptance: all overlay behavior works across normal apps, Spaces, display changes, and app restarts. If the hotkey cannot register, Settings shows the conflict.

7.4 Phase 4: Secure Session and Google Sign-In

Add Google Sign-In through a native browser-based OAuth flow or Google SDK choice fixed in the architecture document. Store refresh/access tokens in Keychain. Add sign-out and token refresh.

Acceptance: a real Mulberry account can sign in, refresh session after app restart, and sign out with all local secrets cleared.

7.5 Phase 5: Bootstrap and Local Domain Model

Implement bootstrap REST client, typed DTOs, domain mapping, pair state, profile state, streak state, and local observable app state. Add schema versioning for local persistence.

Acceptance: the app displays real account, partner, pair, and streak state from the back-end.

7.6 Phase 6: Canvas Operation Model and Renderer

Port the Android canvas operation semantics into Swift models. Implement deterministic application of strokes, point appends, stroke finish, stroke delete, canvas clear, text element add/update/delete, and sticker element add/update/delete. Build the renderer for passive overlay and app canvas.

Acceptance: snapshot fixtures render identically enough across passive overlay and full app, and operation application is covered by unit tests.

7.7 Phase 7: WebSocket Sync and REST Recovery

Implement the WebSocket message set: HELLO, READY, client operation batches, acknowledgements, server operation batches, flow control, error handling, ping/pong, reconnect, and recovery from last applied revision. Queue outbound operations locally.

Acceptance: Mac receives Android operations live, sends operations accepted by the server, reconnects after network loss, and does not duplicate operations after restart.

7.8 Phase 8: Full App Canvas Tools

Implement full app drawing tools first: brush, color, width, eraser, clear, undo, redo, text creation/editing, sticker insertion, and canvas viewport behavior. The full app establishes the canonical tool model used by Quick Draw.

Acceptance: a user can perform the same v1 canvas actions as Android from the full app, and the generated operations sync to Android.

7.9 Phase 9: Quick Draw Editing Parity

Expand Quick Draw to full drawing-canvas feature parity. The Quick Draw palette must include every canvas tool available in the full-size drawing canvas, including brush, color, width, eraser, undo, redo, clear, text creation/editing, sticker insertion, element selection, transform affordances, and Done.

Acceptance: every canvas mutation that can be produced from the full app can also be produced from Quick Draw, with matching operation payloads and sync behavior.

7.10 Phase 10: Stickers, Assets, and Cache

Implement sticker pack catalog client, signed URL loading, disk cache, version-aware invalidation, placeholder rendering, and failure states. Use the same sticker element identifiers as Android.

Acceptance: published sticker packs can be browsed, selected, inserted, cached, and rendered offline once cached.

7.11 Phase 11: Notifications, Login Item, and Background Resilience

Implement login item enablement, visible notifications, reaction handling, pair status notifications, app wake/reconnect behavior, and optional APNs token model behind a feature flag if server support is ready.

Acceptance: after login restart, Mulberry starts in the background, restores overlay, connects when network is available, and shows configured user-visible notifications.

7.12 Phase 12: Packaging, QA, and Release Candidate

Create direct DMG packaging, notarization path, update strategy decision, privacy copy updates, troubleshooting docs, and regression suites. Perform manual QA across macOS versions, Spaces, fullscreen apps, multiple displays, sleep/wake, and network loss.

Acceptance: a signed/notarized DMG installs and runs on a clean Mac, recovers state, syncs with Android, and passes the release checklist.

8 Metrics and Instrumentation

V1 should measure product health without turning the app into an analytics-heavy surface. Required events:

- macOS app installed and first launched;
- sign-in started/succeeded/failed;
- overlay shown/hidden/moved;
- Quick Draw entered/exited and exit reason;
- local operation queued/acked/failed;
- recovery started/completed/failed;
- reaction sent;
- login item enabled/disabled.

Events must not include canvas content, text element content, sticker private URLs, access tokens, refresh tokens, or partner email addresses.

9 Risks

Risk	Impact	Mitigation
------	--------	------------

Desktop layering differences	Overlay may appear below icons/widgets or behave differently across macOS versions.	Promise only below-normal-window passive behavior; test supported versions.
Fullscreen Spaces	Quick Draw cannot reliably appear above another app's fullscreen Space.	Make fullscreen Quick Draw unsupported in v1 and show a prompt.
Canvas parity drift	Swift renderer may diverge from Android rendering.	Use operation fixtures and visual snapshots in both clients.
Multi-device conflicts	Same user may draw from Mac and Android at once.	Rely on server revisions and idempotent client operation IDs.
Hotkey conflicts	Command-Control-M may be taken by another app.	Detect registration failure and expose settings.
Battery/background expectations	Users may expect updates when the app is killed.	State v1 behavior clearly; login item covers normal background use; APNs can follow.

10 Resolved Build Decisions

These decisions are fixed for v1:

1. Final visual design will arrive later; until then, use a barebone functional UI aligned with mobile app parity.
2. Quick Draw has full parity with the full-size drawing canvas and includes every canvas tool.
3. Minimum supported version is macOS 14 or newer; macOS 15 or newer may be selected if implementation needs justify it.
4. V1 distribution is direct DMG.